Response to the committee

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang (2362162), Master's thesis, Electrical Engineering, Polytechnique Montréal

Supervisor: Antoine Lesage-Landry

October 26, 2026

To Sébastien Le Digabel (chair), Antoine Legrain (second member) and Antoine Lesage-Landry (supervisor).

Message au jury

Bonjour chers membres du jury,

Je voulais tout d'abord vous remercier pour vos commentaires sur mon mémoire. Comme vous m'avez demandé de clarifier et d'approfondir plusieurs points des résultats, j'ai pris le temps de les rendre plus complets, et j'ai aussi découvert quelques erreurs dans les affirmations que j'avais faites sur les contributions.

Premièrement, il y a maintenant quatre problèmes qui sont plus lents. J'ai ajouté une section du mémoire pour les décrire (§4.4). Trois des quatre problèmes peuvent devenir plus rapides. Deux d'entre eux le deviennent avec des changements dans le « frontend » (par frontend, je fais référence à la manipulation des arbres d'expression avant le calcul des dérivées). Par exemple, pouvoir détecter que `D @ ones` (où `D` est une matrice quelconque) équivaut à `D.sum(axis=1)`, la somme sur les colonnes, est un grand avantage. Le troisième, Convex-Plasticity, le devient avec un test de densité. Ces changements ne règlent pas tout : une partie du ralentissement vient de `SparseDiffEngine` lui-même (voir Q5).

Deuxièmement, j'ai refait toutes les mesures sur un ordinateur plus récent (MacBook M2 de 2022, avec Apple `Accelerate`), plutôt que sur l'ordinateur Intel de 2018 de la version soumise, pour que les résultats soient reproductibles sur du matériel actuel, comme celui de la plupart des utilisateurs de `CVXPY`. Pour la première canonicalisation, les ratios et les temps sont semblables. Pour les re-résolutions paramétriques, en revanche, `diffengine` n'est plus rapide que sur trois des sept problèmes, contre cinq dans la version soumise. Sur les quatre autres, la passe numérique ne prend que 4 à 24 % d'une re-résolution ; le reste est du post-traitement, après `SparseDiffEngine`. J'ai donc modifié les affirmations de ma conclusion et de la discussion des résultats pour dire seulement qu'il y a des bénéfices, mais que la séparation ne remplace pas encore le tenseur pour les re-résolutions. En refaisant ces mesures avec les versions des logiciels de la version soumise, j'ai vérifié que ce changement ne vient ni de l'ordinateur ni de `SparseDiffEngine`, mais de l'interface entre `CVXPY` et `SparseDiffEngine`, en particulier d'une modification postérieure à la version 1.9.2. Comme les autres backends sont mesurés avec la version 1.9.2, les re-résolutions de `diffengine` sont maintenant mesurées sans cette modification aussi. Réduire ce post-traitement est maintenant le premier des travaux futurs.

Troisièmement, le problème UnconstrainedQP n'était pas réellement 100 fois plus rapide. C'était dû à un changement de frontend que j'avais appliqué sur la branche du backend `diffengine`, mais pas sur celles des autres. En l'appliquant à tous, cet avantage disparaît. Ceci a été rectifié : aucun backend n'utilise maintenant ce changement.

À part ces trois points, qui sont les changements majeurs, j'ai aussi fait les études d'ablation et exécuté plus d'instances de benchmark, avec des mesures de signification statistique, ce qui répond à tous vos commentaires.

J'ai rédigé des ébauches de réponses à vos questions, que Claude, l'outil d'IA déclaré dans le mémoire, a ensuite peaufinées ; les chiffres viennent directement des scripts de mesure.

1 Summary of corrections

I have made all the modifications required by the committee and by Antoine Legrain's examiner report, as well as the minor corrections that Sébastien Le Digabel annotated and those listed in the appendix of Antoine Legrain's report. The minor corrections are not listed here. All changes since the version sent to the committee are marked in `Document-diff.pdf`, with added text in blue and deleted text in red. Section, table and page numbers below refer to the final version.

Table 1 lists the required modifications. "J" refers to the committee's requirements after the defence, and (i)–(vii) to the items of the synthesis of the examiner report. The defence questions are answered in Section 2; the other items are in the thesis sections listed.

Table 1: Required modifications and where they are made.

Request	Change	Where
J1a: statistical significance	30 runs per comparison; Mann–Whitney [1] per problem, Holm-adjusted [2]; Wilcoxon [3] and a bootstrap interval over the suite.	Ch. 4 introduction, p. 51; §4.1
J1b, (ii): ablation	Dense blocks turned off on all 25 problems, in time and memory; the split ablated on the re-solves (Q2).	§4.2, p. 57
J1c: more instances	New parameter values for each run, and every problem scaled in size and density.	Ch. 4 introduction; §4.3
J1d: sensitivity to size and sparsity	All 25 problems at five sizes and eight at seven densities, with and without the dense blocks.	§4.3, p. 60
J1e, (iii): non-DPP problems	Re-solves of two families in DPP, non-DPP and parameter-free variants, and of ConvexPlasticity (Q3).	§4.5.1, p. 66

Request	Change	Where
J2, (i): related literature and DNLP	Matrix-free modelling, `CVXcanon`, `CVXGEN`, `Convex.jl` 0.16, `Pyomo`, block-sparse antecedents and sparse factorization; who leads `SparseDiffEngine` and what is specific to this work (Q6).	§2.2, §1.5, §3.1.1
J3: declaration of AI use	New front-matter page.	Front matter
(iv): repeated measurements	30 runs, each in a fresh process; tables give mean ± standard deviation.	Ch. 4 introduction; Table 4.1
(v): dimensions of the instances	Dimensions, nonzeros, parameter size and DPP status of every problem.	Appendix A, Tables A.1 and A.2
(vi): time by phase	The share of each phase on the four slower problems, with the cause of each.	§4.4, p. 62
(vii): correctness check	The problem data are checked against the `CPP` backend to a relative tolerance of 10^{-9}, at the first canonicalization and after a parameter update.	Ch. 4 introduction

A change of baseline. The version sent to the committee reported a 2.8× geometric-mean speedup of `diffengine` over the best `CVXPY` backend. Two things have changed since. First, all experiments were rerun on an Apple M2 with the current versions of `CVXPY` and `SparseDiffEngine`. Second, the earlier comparison was not fair: `diffengine` was timed with `CVXPY` pull request #3524, which reassociates chains of matrix products that end in a constant, while the other backends were timed without it. That pull request speeds up every backend, so now none of them uses it; likewise, the re-solves of `diffengine` are timed without pull request #3525, a later interface change that only slows `diffengine`. With this baseline, `diffengine` is the fastest of the four backends on 21 of the 25 problems, significantly on all 21, with a geometric-mean ratio of 0.44 to the best `CVXPY` backend, about 2.3× faster. UnconstrainedQP is now 11.7× slower (§4.4, p. 62). Every timed comparison is made in 30 runs, each in a fresh process, on an Apple M2 with Apple's `Accelerate`, single-threaded.

2 Replies to the defence questions

Q1. *Why materialize dense blocks rather than leave them as operators, as in the matrix-free approach of Diamond and Boyd [4]? When is each approach preferable?*

Because the solvers that `CVXPY` calls need the matrices. Its default solvers are `Clarabel`, an interior-point method, and `OSQP` and `SCS`, which are first-order methods; commercial solvers such as `MOSEK` and `Gurobi` are interior-point or simplex methods. All of them factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$: an interior-point method factorizes a KKT system at every iteration, and `OSQP` and `SCS`, in their default direct mode, factorize their KKT matrix once

and reuse the factorization across iterations. So every one of them needs $\mathbf{A}$ and $\mathbf{P}$ explicitly, whatever its algorithm. Matrix-free modelling pays off for very large problems built from fast transforms, solved by first-order methods that only need products with $\mathbf{A}$ and $\mathbf{A}^\top$, such as the indirect mode of `SCS`. A permuted dense block lies between the two: it is a materialized matrix, but it keeps the structure of the operator, namely its row and column selectors and its dense block.

Changes in the thesis, §2.2, p. 24

In convex optimization, Diamond and Boyd take the matrix-free view. Instead of forming $\mathbf{A}$, they represent the problem as a graph of linear operators, available only through their forward and adjoint products, which suits first-order solvers. The solvers that `CVXPY` calls by default instead factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$. The matrix-free approach pays off on very large problems built from fast transforms, such as convolutions and Fourier transforms, and it needs solvers that accept operators instead of matrices. The permuted dense matrices of §3.2 sit between the two: they form the matrix that the solver factorizes, but keep the structure of the operator, its row and column selectors and its dense block.

Q2. *If the permuted dense representations are disabled, what fraction of the speedup remains?*

Most of it. Without the dense blocks, `diffengine` is 2.15× faster than the best `CVXPY` backend in geometric mean, against 2.29× with them, and the difference is not significant over the suite (Wilcoxon, $p = 0.07$). Per problem, however, the effect is large in both directions (Figures 1 and 2): the dense blocks help by 1.8 to 3× on eight problems with dense data, and hurt by 1.4 to 46× on the three that build a dense block from sparse data (CVaR, ConvexPlasticity, UnconstrainedQP). Without them, `diffengine` also uses 0.83 times the peak memory in geometric mean.

The dense blocks are the only part of the engine that can be turned off on a first canonicalization; the implementation in C, one Jacobian per node and the global layout are built into every atom, so we do not attribute the remaining 2.15× to any one of them. The split between the symbolic and numeric passes matters only on re-solves, where removing it makes a re-solve 2.9× slower, significantly on all seven problems (Table 2). *We conclude that most of the first-canonicalization speedup comes from the engine as a whole rather than from the dense blocks, which should only be built from data that is dense in fact, and that the re-solve speed comes from the split.*

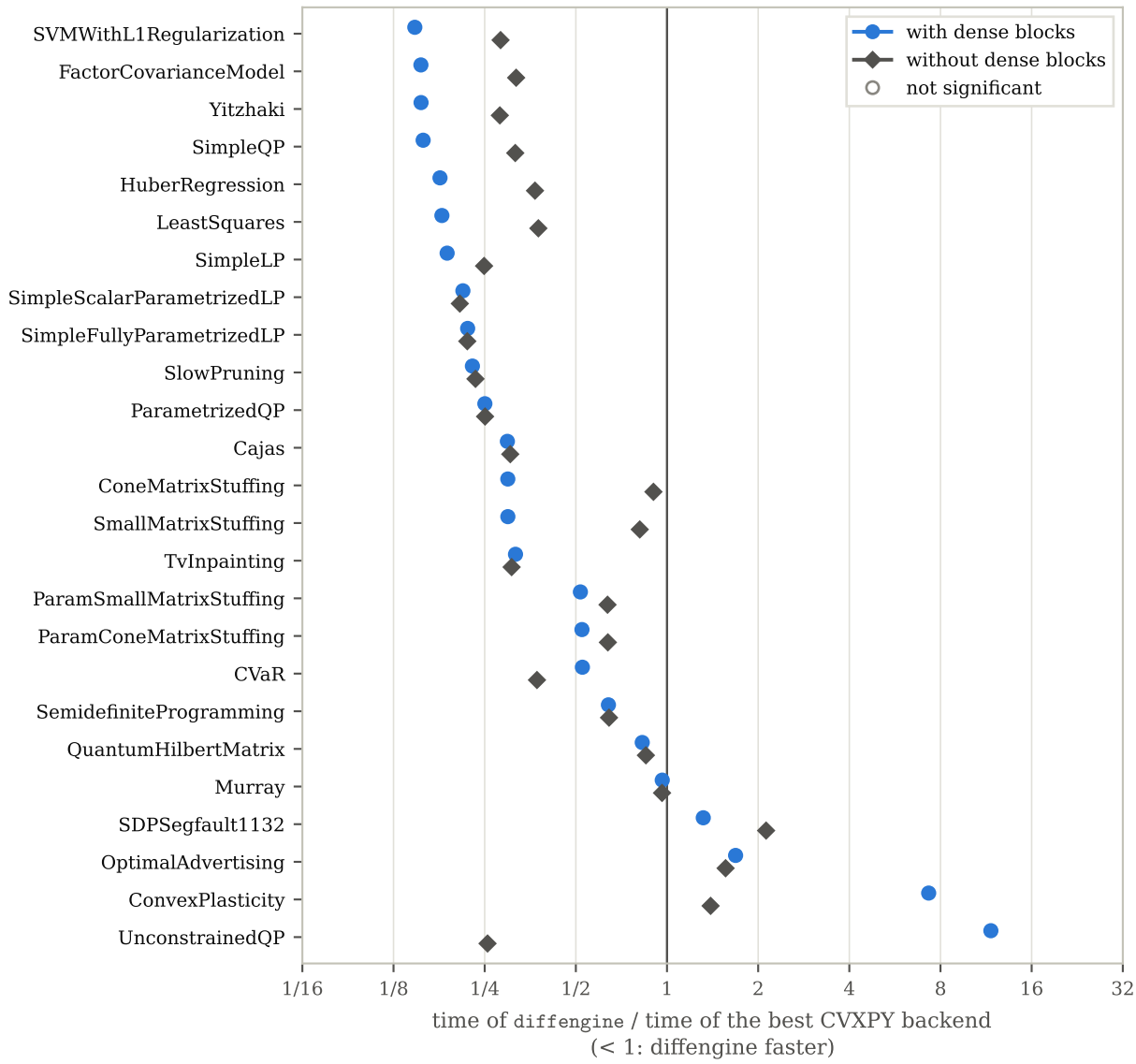


Figure 1: Figure 4.1 of the thesis (p. 55): ratio of `diffengine` to the best `CVXPY` backend on each problem, with its bootstrap 95 % confidence interval, with and without the dense blocks. Filled markers are significant (Mann–Whitney, Holm-adjusted $p < 0.05$); here every one is.

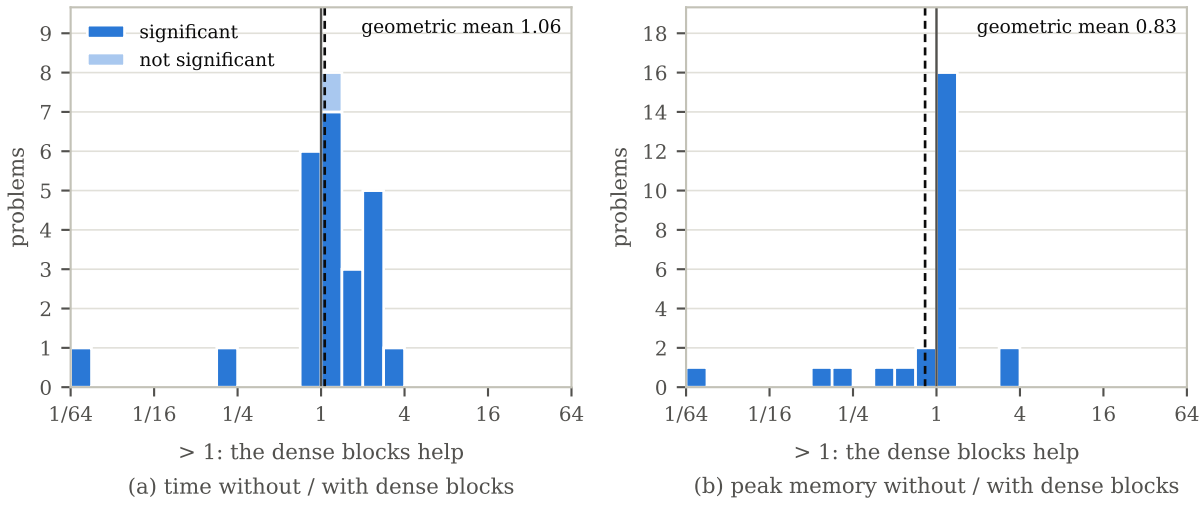


Figure 2: Figure 4.3 of the thesis (p. 58): distribution over the 25 problems of the ratio without / with the dense blocks, in time (a) and peak memory (b). The dashed line is the geometric mean.

Table 2: The ablations in summary: geometric mean of the ratio of each variant to its reference, its bootstrap 95 % confidence interval, the Wilcoxon p-value over the problems, and the number of problems with a significant difference (Mann–Whitney, Holm-adjusted).

Variant	Reference	Ratio	95 % CI	p	Significant
First canonicalization, 25 problems					
no dense blocks	`diffengine`	1.06	0.71–1.45	0.067	24/25
no dense blocks	best `CVXPY`	0.46	0.36–0.61	3.2×10^{-5}	25/25
Re-solve, 7 DPP problems					
no dense blocks	`diffengine`	1.01	0.98–1.05	0.938	6/7
no split	`diffengine`	2.86	1.73–4.91	0.016	7/7
neither	`diffengine`	3.68	2.19–6.20	0.016	7/7

Q3. *How does re-solving behave on a non-DPP parametric problem?*

On the two families written with a product of two parameters, which DPP forbids, `CVXPY` has to canonicalize the problem again at every solve, while `diffengine` keeps its symbolic pass and reruns only the numeric pass. It re-solves as fast as on the DPP variant, 13 to 14× faster than `CVXPY` on `lasso` and `factor_cov`, significantly on both (Table 3). ConvexPlasticity, the suite's only natural non-DPP parametric problem, is the exception: it re-solves in 207 ms against 42 ms for `CVXPY`, because 88 % of the re-solve assembles a dense 3000×3000 block of $\mathbf{P}$ built from a diagonal. Without the dense blocks, it re-solves in 2.0 ms, 21× faster than `CVXPY`. The thesis no longer claims that the split replaces the tensor (Q4).

Table 3: Table 4.7 of the thesis (p. 67): re-solve time in milliseconds, mean over 30 runs. `CVXPY` applies the tensor on the DPP variant and canonicalizes again on the others; the cached column of `diffengine` runs the numeric pass only. The ratio is the median of `diffengine`, cached when it can, over that of `CVXPY`.

			`diffengine`		
Family	Variant	`CVXPY`	cached	rebuild	Ratio
`lasso`, $n = 1000$	DPP	6.4	23.3	32.7	3.64*
	non-DPP	315	23.3	32.8	0.07*
	parameter-free	338	—	49.0	0.14*
`factor_cov`, $n = 5000$	DPP	1.2	3.6	6.9	2.86*
	non-DPP	45.8	3.6	7.0	0.08*
	parameter-free	47.1	—	11.0	0.23*

Changes in the thesis, §4.5.1, p. 66

On the non-DPP variants, `CVXPY` canonicalizes the problem again at every solve, while `diffengine` keeps its symbolic pass and reruns only the numeric pass, so it re-solves as fast as on the DPP variant, 13 to 14 times faster than `CVXPY`. On the DPP variants, the tensor is 3.6 and 2.9 times faster. ConvexPlasticity, the only parametric problem of the suite that is not DPP, is the exception: `diffengine` re-solves it in 207 milliseconds against 42 for `CVXPY`, because 88 % of the re-solve assembles the dense 3000×3000 block of $\mathbf{P}$. Without the dense blocks, it re-solves in 2.0 milliseconds, 21 times faster than `CVXPY`.

Changes in the thesis, §4.6, p. 67

Caching canonicalization does not necessarily require the DPP ruleset. [...] the symbolic and numeric split re-solves within a factor of four of the cached tensor on every DPP problem. [...] it re-solves problems that are not DPP, on which `CVXPY` has to canonicalize again, 13 to 14 times faster.

Q4. *Why not re-evaluate only the subtrees that depend on the updated parameters?*

The engine now does this: after a parameter update, the numeric pass re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others. All re-solve times in the thesis use it.

On the seven DPP problems (Table 4), `diffengine` has the fastest first canonicalization on all seven, 2.5 to 6.6× faster than the fastest tensor build. The cached tensor re-solves faster on four of them, by 1.10 to 3.60×, and recovers its slower first canonicalization only after 18 to 124 re-solves. On those four, the numeric pass takes only 4 to 24 % of a re-solve; the rest is post-processing after the engine, the conversion to the solver's format. *We conclude that the split does not replace the tensor on DPP problems that are re-solved many times, but comes within a factor of four of it without the DPP rules.* Making the post-processing cheaper is now the first item of future work.

Table 4: Table 4.5 of the thesis (p. 65): first canonicalization in seconds and re-solve in milliseconds, mean over 30 runs. A dash is a tensor that exceeded the memory or time limit. The ratio is the median re-solve of `diffengine` over that of the fastest tensor, * when significant.

	first canonicalization (s)				re-solve (ms)		
Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	tensor	`diffengine`	Ratio
ParamSmallMatrixStuffing	2.02	7.30	0.083	**0.033**	1.8	**0.4**	0.24*
ParamConeMatrixStuffing	0.74	1.85	0.055	**0.022**	1.0	**0.3**	0.27*
SimpleFullyParametrizedLP	—	—	0.57	**0.12**	47	**35**	0.73*
SimpleScalarParametrizedLP	1.33	1.17	1.19	**0.23**	**63**	70	1.10*
ParametrizedQP	—	—	2.63	**0.76**	**299**	349	1.16*
SVMWithL1Regularization	2.80	2.57	2.83	**0.39**	**38**	125	3.26*
FactorCovarianceModel	1.20	1.14	1.19	**0.18**	**21**	74	3.60*

Changes in the thesis, §3.4.1, p. 47

The numeric pass therefore skips an affine tree after the first evaluation, until the next parameter update. After an update, it re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others.

Q5. *When does the global layout become a handicap relative to `CVXPY`'s local layout, in particular for memory?*

When the problem has many variables and many nodes. In the global layout, each node keeps, besides its Jacobian, integer work arrays of length n, the total number of variables, whatever variables its own subtree contains. Their memory is therefore $O(\text{nodes} \times n)$ integers. In `CVXPY`'s local layout, each node stores blocks only for the variables it depends on, so the same overhead is proportional to the sum, over the nodes, of the number of variables each one depends on. The two coincide when every node depends on every variable, and the local layout wins when most nodes depend on a few variables out of many.

OptimalAdvertising shows it. Each of its 250 minimum terms depends on one row of a 250×1000 variable, but has a Jacobian over all 250 000 variables. Without the dense blocks, its peak memory is 945 MB, against 173 to 197 MB for the `CVXPY` backends. With them it reaches 6.1 GB, because each dense block of the aggregate constraints $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ and $\mathbf{D}\mathbf{1} \geq \mathbf{c}$ also keeps one integer per variable of the problem: the same cost, paid per block rather than per node. Rewriting these products with a vector of ones as sums avoids the dense blocks:

OptimalAdvertising then canonicalizes in 0.33 s instead of 0.91 s, faster than the 0.53 s of `CVXPY`, with 1.4 GB instead of 6.1 GB. Storing the per-block maps sparsely would fix the general case.

Changes in the thesis, §5.1, p. 69

Another part of the memory cost comes from the global layout. Every node of the expression trees holds a Jacobian over all n variables, together with integer work arrays of length n. The values stored remain proportional to the nonzeros, but this overhead grows with the number of nodes times the number of variables. On problems with very many variables and deep trees, it can exceed the memory of `CVXPY`'s local layout, whose blocks span only the variables that each node depends on.

Q6. *What part of* ***`SparseDiffEngine`*** *pre-existed the thesis or was developed for DNLP, and what part is specific to this work?*

Daniel Cederberg leads `SparseDiffEngine`. He started it as the differentiation engine of the `DNLP` extension of `CVXPY` [5], which differentiates nonlinear atoms at every iterate of the solver, and I co-developed it with him. The version sent to the committee should have said so clearly; it now does. This thesis uses the engine for a different purpose, canonicalization: it evaluates the affine atoms and the quadratic form at the origin to obtain the problem data. What is specific to the thesis is the interface to `CVXPY`, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Changes in the thesis, §1.5, p. 16

Daniel Cederberg is the lead developer of `SparseDiffEngine`. It was first built as the differentiation engine of the `DNLP` extension of `CVXPY`, where it differentiates nonlinear atoms such as `exp`, `log` and `entr` at every iterate of the solver. This thesis builds on the observation that canonicalization is a special case of the same computation: once a problem is canonicalized, only affine atoms and quadratic forms remain, and their derivatives are the problem data.

Changes in the thesis, §3.1.1, p. 37

What is specific to this thesis is the interface to `CVXPY` of §3.3, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Q7. *What is the strategy for BLAS and binary distribution of* ***`sparsediffpy`****?*

`cvxcore` is a C++ extension that lives inside the `CVXPY` repository, so `CVXPY` has to compile and ship it. `sparsediffpy` is a separate package that `CVXPY` only depends on. Its compiled wheels for Linux, macOS and Windows are published on `PyPI`; they link Apple's `Accelerate` on macOS and bundle `OpenBLAS` elsewhere, so users install nothing beyond the package. The thesis now says so.

Changes in the thesis, §3.4, p. 47

Unlike `cvxcore`, which is compiled inside the `CVXPY` repository, `sparsediffpy` is a separate package on which `CVXPY` only depends. Its compiled wheels for Linux, macOS and Windows

link Apple's `Accelerate` on macOS and a bundled copy of `OpenBLAS` on the other two, so users need no BLAS of their own.

References

[1] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[2] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[3] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[4] S. Diamond and S. Boyd, "Matrix-free convex optimization modeling," in *Optimization and Its Applications in Control and Data Sciences*, ser. Springer Optimization and Its Applications, B. Goldengorin, Ed. Springer, 2016, vol. 115, pp. 221–264.

[5] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," arXiv:2606.02896, 2026. [Online]. Available: https://arxiv.org/abs/2606.02896